

FDSReader: A Python package for fire dynamics simulation data analysis

Jan Vogelsang^{1,2¶}, Marc Fehr^{3,4}, Kristian Börger^{3,4}, and Lukas Arnold^{3,4}

¹ Neuromorphic Software Ecosystems (PGI-15), Jülich Research Centre, Jülich, Germany ² RWTH Aachen University, Aachen, Germany ³ Institute for Advanced Simulation (IAS-7), Jülich Research Centre, Jülich, Germany ⁴ Chair of Computational Civil Engineering, University of Wuppertal, Wuppertal, Germany ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 12 May 2026

Published: unpublished

License

Authors of papers retain copyright⁸ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

In recent years, Python has become the most popular programming language in the scientific community. It offers extensive functionality for preparing, post-processing and visualising data of all types and origins. Through a large number of freely available packages, the programming environment can be flexibly adapted to a researcher's individual needs. Up until now fire simulation data as produced by the Fire Dynamics Simulator (FDS) (McGrattan & McDermott, 2017) for example could not be easily imported and processed with Python. Although Smokeview (Forney, 2017) enabled the visualization of simulation results, it has limited functionality for conducting detailed quantitative analyses of the generated data. We publish the *FDSReader*, an open-source Python package to extract and analyze data generated by FDS simulations in a straightforward "Pythonic" and efficient way.

Statement of need

At present, most working groups in both industry and research have no unified solution for efficiently extracting the binary data output of FDS in Python for further processing. This is a common requirement for both academic research and fire safety engineers who need to analyze data precisely and not rely on vague estimates via the GUI Smokeview provides.

State of the field

Currently, there are two possible ways to analyze data generated by FDS: *Fds2Ascii*, a Fortran program to manually convert specific binary output files of FDS into a text-based format and subsequently read in the text file with the preferred programming language. Each FDS simulation outputs many of such binary files, sometimes hundreds or even thousands, depending on the size of the simulation. Converting all files is cumbersome, and interpreting the resulting text outputs remains non-trivial. Alternatively, researchers write custom scripts to read the binary data directly. Consequently, many groups maintain their own fragmented sets of (typically Python-based) scripts for various data types (e.g., slices, boundary data). Because the internal data structures of FDS are highly complex, these custom scripts are time-consuming to develop, often restricted to narrow simulation parameters (e.g., single-mesh only), and prone to breaking with new FDS releases. Therefore, rather than attempting to wrap the inefficient Fortran-based *Fds2Ascii* pipeline or maintain a fragmented ecosystem of limited custom scripts, developing a native, comprehensive, and actively maintained Python package from scratch was strictly necessary.

40 Software design

41 The *FDSReader* Python package provides functionality to automatically import the full set
42 of FDS outputs for both old and new versions of FDS. When first loading a simulation, all
43 metadata is collected and cached to reduce subsequent load times. The complete data produced
44 during the simulation - such as the slice data for each time step - is lazy-loaded, i.e., only fully
45 loaded from disk when the data is accessed by the user via Python. The first access triggers
46 loading the data into memory in the background and converting it to equivalent Python data
47 structures. This method minimizes initial loading time without sacrificing the ability to easily
48 filter and select the desired data. The *FDSReader* collects all data of the same type into one
49 collection and offers its own set functions for each type to easily select the data which should
50 be further processed. These functions can even be called in an automated fashion, to run some
51 predefined post-processing routines on simulation data without having to manually interact
52 with the data, as one would have to do when using *SmokeView* or *Fds2Ascii*. The *FDSReader*
53 is able to read all data types available in FDS including most of the additional metadata. To
54 be concrete, the package contains modules for slices (*slcf*), boundary data (*bndf*), volumes
55 (*plot3d*, *smoke3d*) particles (*part*), isosurfaces (*isof*), evacuations (*evac*), complex geometry
56 boundary data (*geom*), devices (*devc*) and meshes. Slices, boundary data, isosurfaces and
57 volumes are each respectively separated into multiple parts, one for each mesh. While FDS
58 outputs the data separately for each mesh, the *FDSReader* provides methods to combine all
59 these parts automatically and operate on data across meshes.

60 The following code snippet demonstrates the use of *FDSReader* to load a temperature slice
61 file from an FDS simulation. For the simulation times $t = 500$ s, the data are stored as a
62 two-dimensional NumPy array. This array can, for example, be visualized using the *imshow*
63 function of the *matplotlib.pyplot* module.

```
import fdsreader as fds

sim = fds.Simulation('path/to/simulation')
slcf = sim.slices.filter_by_quantity('TEMPERATURE').get_by_id('SLCF_TEMP_1')
time_index = slcf.get_nearest_timestep(500)
data = slcf.to_global()[time_index]
```

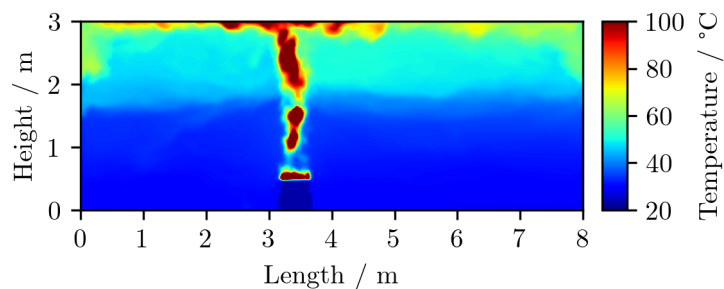


Figure 1: Visualization of an *fds* slice file at timestep $t = 500$ s via the *imshow* function from the *matplotlib.pyplot* module. The data was read using the *FDSReader* module

64 Research impact statement

65 Since the first release of the *FDSReader* in 2020, the package received widespread attention.
66 According to ClickHouse download metrics ([ClickHouse, 2026](#)), the package has been
67 downloaded over 263,000 times. It is supported by an active community, with issues and pull
68 requests regularly submitted by both the original authors and external developers. Furthermore,
69 the *FDSReader* ecosystem has spurred downstream development, leading to external forks and

70 the creation of adjacent tools, such as an open-source web application interface (Balabomma,
71 2026).

72 AI Usage Disclosure

73 No generative AI was used for architectural design or core algorithms. Claude (Anthropic)
74 was used for creating documentation and docstrings, and for assisting in maintenance tasks
75 (resolving issues). Generative content was thoroughly reviewed and validated.

76 References

- 77 Balabomma. (2026). *FDSreader_webapp: A web interface for FDSReader*. https://github.com/Balabomma/FDSreader_webapp
- 78
- 79 ClickHouse. (2026). *FDSReader download metrics dashboard*. [https://clickpy.clickhouse.com/](https://clickpy.clickhouse.com/dashboard/fdsreader)
80 [dashboard/fdsreader](https://clickpy.clickhouse.com/dashboard/fdsreader)
- 81 Forney, G. P. (2017). *Smokeview, a tool for visualizing fire dynamics simulation data, volume i: User's guide, sixth edition* [Special Publication (NIST SP)]. National Institute of Standards;
82 Technology. [https://github.com/firemodels/fds/releases/download/FDS-6.10.1/SMV_](https://github.com/firemodels/fds/releases/download/FDS-6.10.1/SMV_User_Guide.pdf)
83 [User_Guide.pdf](https://github.com/firemodels/fds/releases/download/FDS-6.10.1/SMV_User_Guide.pdf)
- 84
- 85 McGrattan, K., & McDermott, R. (2017). *Fire dynamics simulator user's guide, sixth edition*
86 [Special Publication (NIST SP)]. National Institute of Standards; Technology. <https://doi.org/10.6028/NIST.SP.1019>
87